EXPERIMENT - 60

1. Problem Statement

Write a C program that reads two strings, s and t, both containing only lowercase characters. Pass them to a function named check_anagram() using their addresses. The function must:

- Determine whether s and t contain the same characters with the same frequencies.
- Print "Anagram" if they are valid anagrams, and "Not Anagram" if they are not.
- Treat the order of characters as irrelevant.
- Consider strings of different lengths as not anagrams.
- Use a character-frequency-counting approach.

2. Algorithm

Main Program

Step 1: START
Step 2: Read the strings s and t.
Step 3: Call check_anagram(s, t).
Step 4: STOP

Function check_anagram()

Step 1: Set counts and countt to 0, and lens = 0 and lent = 0.
Step 2: For every character of s: add 1 to counts[s[i] − 'a'] and add 1 to lens.
Step 3: For every character of t: add 1 to countt[t[i] − 'a'] and add 1 to lent.
Step 4: If lens is not equal to lent, print "Not Anagram" and return.
Step 5: For i = 0 to 25: if counts[i] is not equal to countt[i], print "Not Anagram" and return.
Step 6: Print "Anagram" and return.

3. Test Case Table

#	s	t	Expected Output	Actual Output	Result
1	listen	silent	Anagram	Matches example 1 in the question	**PASS**
2	hello	world	Not Anagram	Matches example 2	**PASS**
3	anagram	nagaram	Anagram	Matches example 3	**PASS**
4	rat	car	Not Anagram, as c and t do not match up	Matches example 4	**PASS**
5	aab	abb	Not Anagram, since the letters match but the counts do not	Not Anagram, so frequencies really are being compared	**PASS**

#	s	t	Expected Output	Actual Output	Result
6	abc	abcd	Not Anagram, the lengths differ	Caught by the length test	**PASS**
7	abc	abc	Anagram, a string against itself	Anagram	**PASS**
8	aabbcc	abcabc	Anagram, same letters twice each in a different order	Anagram, so the order was ignored	**PASS**

4. Source Code

anagram60.c

```
/* Experiment 42. One function for each of the three operations.
   Every option reads its own input here, which is different from what
   I have been doing. Option 3 needs two strings and the other two need
   one, so there was no single string I could read at the start that
   would have suited all three.
   The question lists only three operations, but a do while menu has no
   way of stopping, so option 4 is mine. Without it the menu keeps
   coming back and the only way out is to kill the program. */

#include <stdio.h>

void removeVowels(const char s[], char result[]);
char firstRepeatingLowercase(const char s[]);
int areAnagrams(const char s1[], const char s2[]);

int main(void)
{
    char str[100], str2[100], result[100];
    char ch;
    int choice;

    do {
        printf("\nMENU\n");
        printf("1. Remove all the vowels from a string\n");
        printf("2. Find the first repeating lowercase letter\n");
        printf("3. Check whether two strings are anagrams\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter a string: ");
                scanf(" %[^\n]", str);
                removeVowels(str, result);
                printf("String without vowels: %s\n", result);
                break;

            case 2:
                printf("Enter a string: ");
                scanf(" %[^\n]", str);
                ch = firstRepeatingLowercase(str);
                if (ch == '\0')
                    printf("No lowercase letter repeats in this string\n");
                else
                    printf("First repeating lowercase letter: %c\n", ch);
                break;

            case 3:
                printf("Enter the first string: ");
                scanf(" %[^\n]", str);
                printf("Enter the second string: ");
                scanf(" %[^\n]", str2);
                if (areAnagrams(str, str2))
                    printf("The two strings are anagrams\n");
                else
                    printf("The two strings are not anagrams\n");
                break;

            case 4:
                printf("Exiting the program.\n");
                break;
        }
    } while (choice != 4);

    return 0;
}
```

```
/* i walks through the old string and j marks where to write in the new
   one, and they drift apart because j only moves when something is
   actually copied.
   The '\0' on the last line is the one I forgot. Programming in C came
   out right and then had junk stuck on the end of it, because printf
   had no idea where the new string finished.
   Both cases are listed because A and a are different characters, and
   leaving the capitals out would have let the A through in Day. */
void removeVowels(const char s[], char result[])
{
    int i, j = 0;
    char ch;

    for (i = 0; s[i] != '\0'; i++) {
        ch = s[i];

        if (ch != 'a' && ch != 'e' && ch != 'i' && ch != 'o' && ch != 'u' &&
            ch != 'A' && ch != 'E' && ch != 'I' && ch != 'O' && ch != 'U') {
            result[j] = ch;
            j++;
        }
    }

    result[j] = '\0';
}

/* First repeating can be read two ways, and the two give different
   answers. In abcdbca one reading says b, the first letter you meet
   that you have seen before. The other says a, because out of the
   letters that repeat at all, a is the one that showed up earliest.
   I went with the second, which is the usual meaning, and that is why
   the counting happens first and the walk from the left happens after.
   One single pass would have given the other answer.
   count has 26 boxes, one per small letter, and s[i] - 'a' turns a
   letter straight into its box number, since 'a' is 97 and 'b' is 98.
   Capitals are skipped in both loops, so AABBcdc answers c, even though
   A and B repeat more times than c does. */
char firstRepeatingLowercase(const char s[])
{
    int count[26] = {0};
    int i;

    for (i = 0; s[i] != '\0'; i++)
        if (s[i] >= 'a' && s[i] <= 'z')
            count[s[i] - 'a']++;

    for (i = 0; s[i] != '\0'; i++)
        if (s[i] >= 'a' && s[i] <= 'z' && count[s[i] - 'a'] > 1)
            return s[i];

    /* '\0' is not a letter, so main can use it to mean nothing was
       found. */
    return '\0';
}

/* Anagrams hold the same characters the same number of times, only in
   a different order. My first idea was to sort both strings and see if
   they came out identical, but counting turned out to be less code, and
   it leaves the two strings alone instead of rearranging them.
   256 boxes covers every character there is, so spaces and digits are
   counted along with the letters, and nothing is quietly ignored.
   The length test is not only a shortcut. The counts could agree on
   every character and the strings still be different lengths if one of
   them held something the other did not. */
int areAnagrams(const char s1[], const char s2[])
{
    int count1[256] = {0};
    int count2[256] = {0};
    int i, len1 = 0, len2 = 0;

    for (i = 0; s1[i] != '\0'; i++) {
        count1[(unsigned char) s1[i]]++;
        len1++;
```

```
    }

    for (i = 0; s2[i] != '\0'; i++) {
        count2[(unsigned char) s2[i]]++;
        len2++;
    }

    if (len1 != len2)
        return 0;

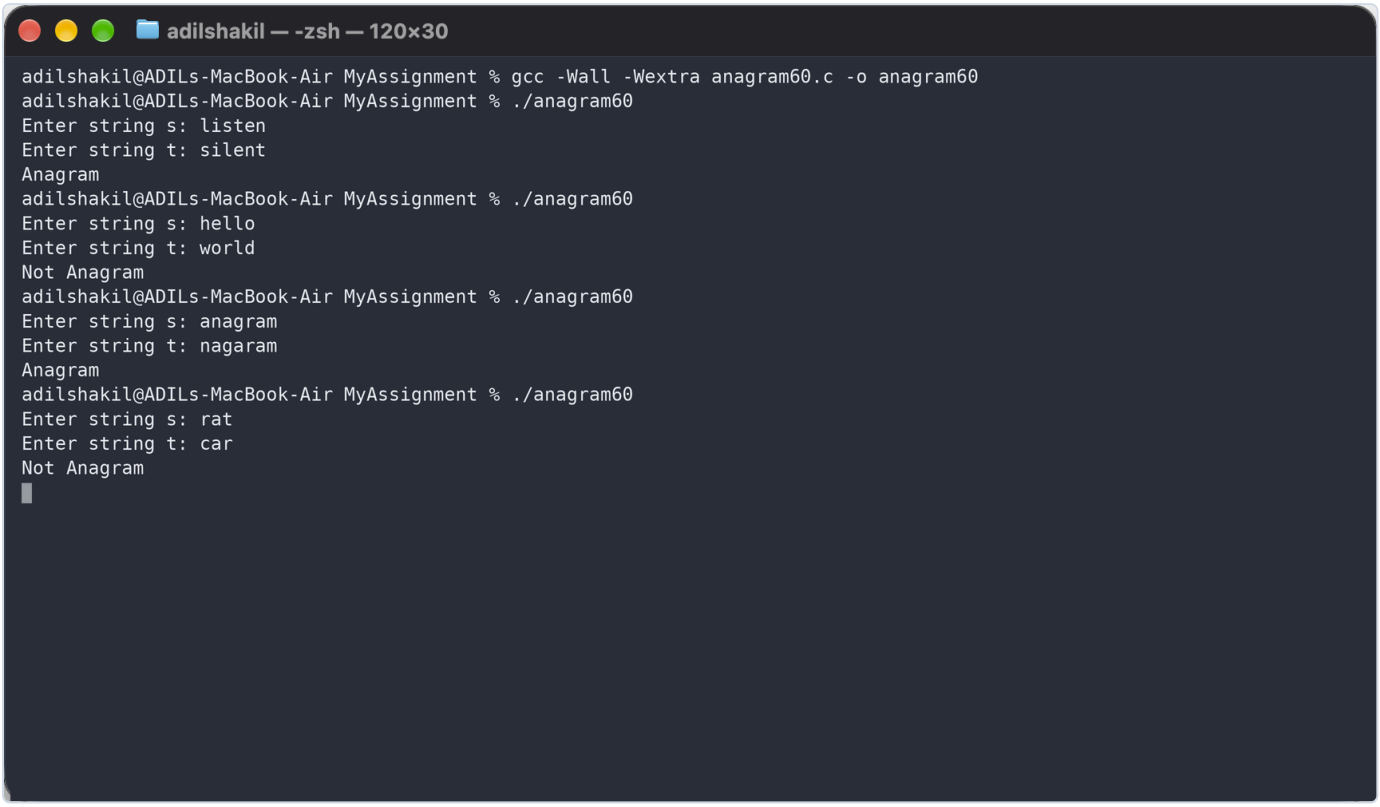
    for (i = 0; i < 256; i++)
        if (count1[i] != count2[i])
            return 0;

    return 1;
}
```

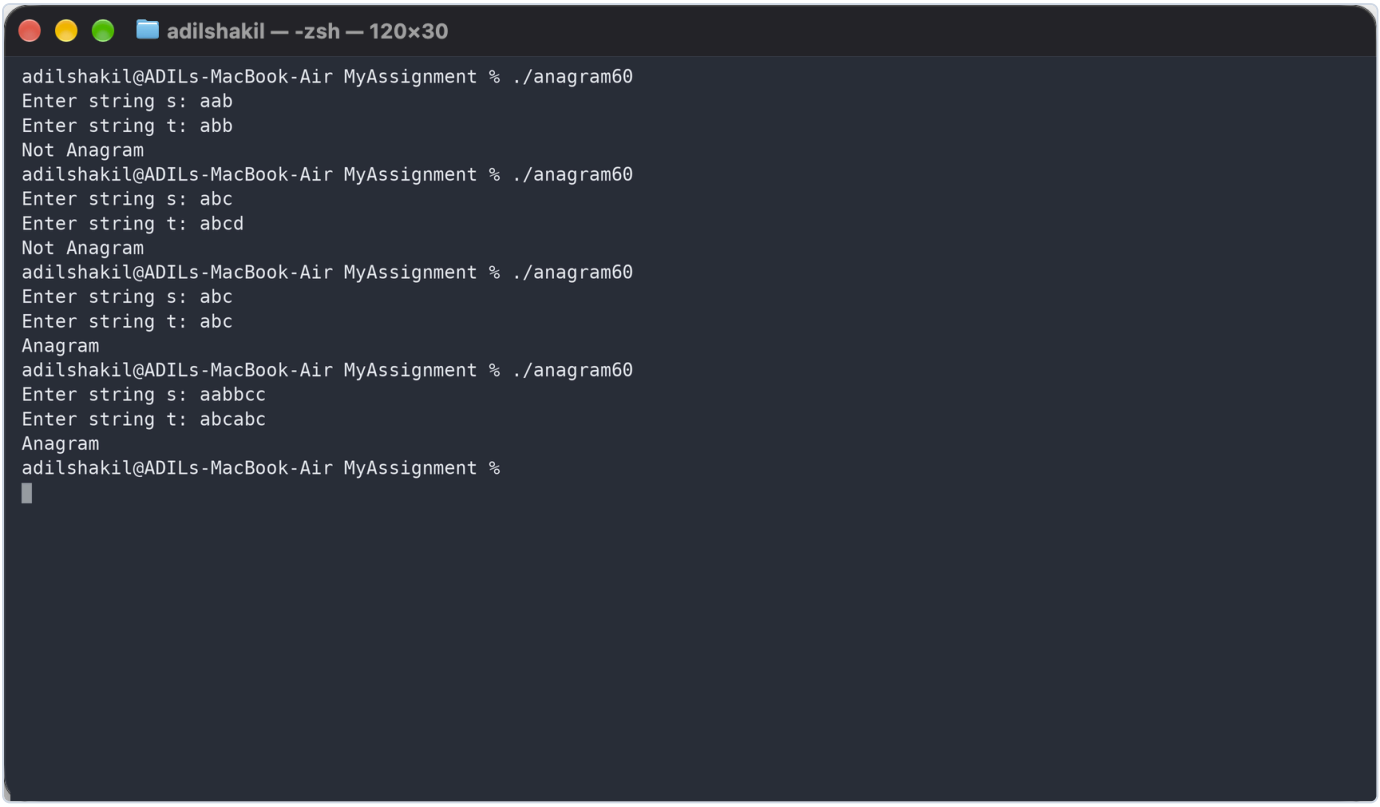
5. Compilation and Output

Compiled with `gcc -Wall -Wextra anagram60.c -o anagram60` so the small warnings would show up too. It built clean.

The Four Examples from the Question



Same Letters with Different Counts, Different Lengths, and Two More



All eight cases came out right, and the first four are the examples from the question.

Each string gets 26 boxes, one per lowercase letter, and `s[i] - 'a'` turns a letter into its box number. 26 is enough only because the input is promised lowercase. Counting also throws the order away by itself, which is exactly what an anagram needs.

Case 5 is the one I added for the word frequencies. aab and abb use the same two letters, so a check that only asked which letters appear would call them anagrams. Comparing the counts catches it. The length test matters for the same reason, since abc and abcabc share every letter.